hashlock.

# Security Audit

## Miracle World

January 2026 info@hashlock.com.au Hashlock Pty Ltd

# Table of Contents

## CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

# Executive Summary

The Miracle World team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

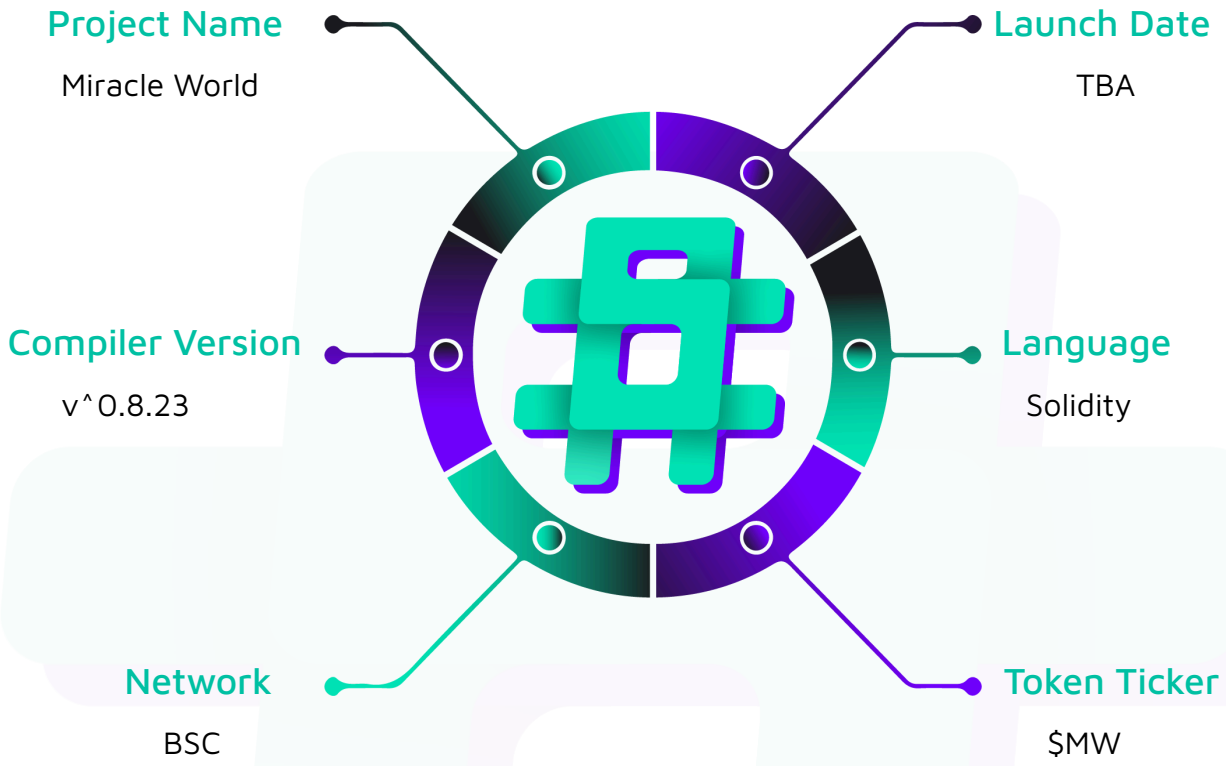
# Project Context

MW Token delivers world-class digital infrastructure built for the demands of modern enterprises and decentralized ecosystems. Traditional enterprise software and financial systems are often fragmented, expensive, and slow to adapt. MW Token solves these challenges by unifying enterprise-grade software with blockchain-powered economics.

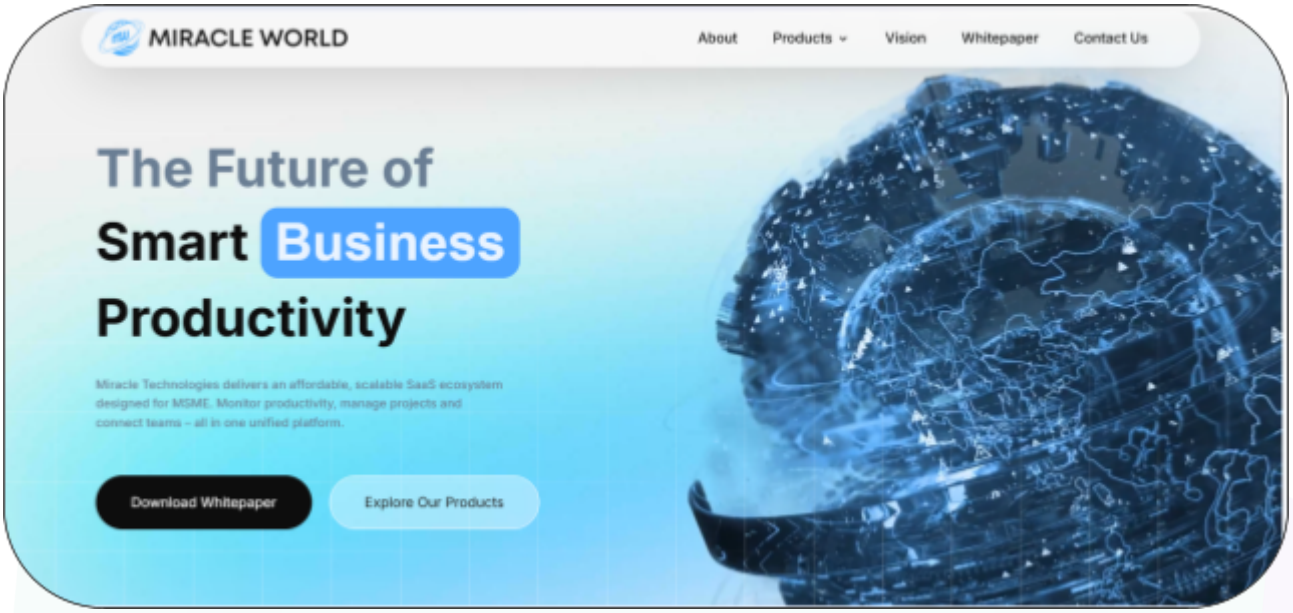
The ecosystem integrates automation, CRM, ERP, and workflow management into a single scalable platform while introducing a token-based payment model. Businesses that purchase corporate solutions using MW Token will receive a discount of up to 25%, lowering the cost of ownership, increasing ROI and lowering total cost of ownership.
MW Token is powering the future of business automation, enterprise software, digital transformation, and next-generation SaaS ecosystems on the blockchain.

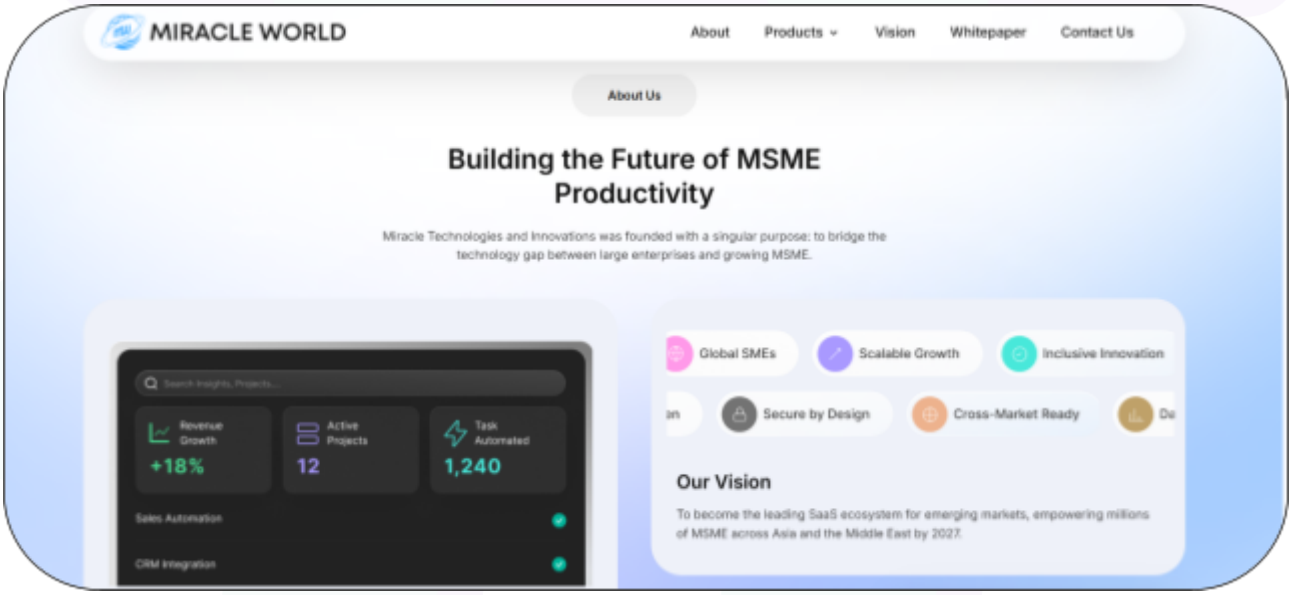
**Project Name**: Miracle World
**Project Type**: Wallet, DeFi
**Compiler Version**: ^0.8.23
**Website**: https://mwtoken.io/
**Logo**:

**Visualised Context:**



**Project Visuals:**



# Audit Scope

We at Hashlock audited the solidity code within the Miracle World project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

| **Description** | **Miracle World Smart Contracts** |
|---|---|
| **Platform** | **Binance** |
| **Audit Date** | **January, 2026** |
| **Contract 1** | MiracleWorld.sol |
| **Audited GitHub Commit Hash** | b8ac97a2519c61f3a6a198f660e8e629c648c65a |
| **Fix Review GitHub Commit Hash** | 45cb06c2d9cfe5e69f5171862a8c31851e5a4984 |

# Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.


*The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.*

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

**Hashlock found:**

4 Medium severity vulnerabilities

3 Low severity vulnerabilities

**Caution:** *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

# Intended Smart Contract Functions

| Claimed Behaviour | Actual Behaviour |
| --- | --- |
| **MiracleWorld.sol**<br>- Allows the operator to:<br>  - Set operator address<br>  - Perform an emergency withdrawal<br>- Allows the system authority to:<br>  - Claim unresolved pools<br>- Allows users to:<br>  - Create prediction pools<br>  - Buy shares<br>  - Sell shares<br>  - Resolve pools<br>  - Claim winnings<br>  - Claim unresolved pools | **Contract achieves this functionality.** |

# Code Quality

This audit scope involves the smart contracts of the Miracle World project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

# Audit Resources

We were given the Miracle World project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

# Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

# Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

| Significance | Description |
|---|---|
| **High** | High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community. |
| **Medium** | Medium-level difficulties should be solved before deployment, but won't result in loss of funds. |
| **Low** | Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future. |
| **Gas** | Gas Optimisations, issues, and inefficiencies. |
| **QA** | Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code. |

# Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

| Significance | Description |
|---|---|
| **Resolved** | The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue. |
| **Acknowledged** | The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception. |
| **Unresolved** | The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed. |

# Audit Findings

## Medium

### [M-01] MiracleWorld#freeze() - Freeze cap can be exceeded due to pre-state-only limit check

**Description**

The freeze cap is not enforced correctly because the contract checks only the current `totalFreezed` value and does not validate the post-freeze value..

**Vulnerability Details**

In MiracleWorld.sol contract, `freezingLimitNotReached` checks only `totalFreezed < FREEZE_LIMIT`. In `freeze()`, the contract then executes `totalFreezed += _amount` without checking `totalFreezed + _amount <= FREEZE_LIMIT`. If `totalFreezed` is just below the limit, a single `freeze()` call can push `totalFreezed` above `FREEZE_LIMIT`, meaning the cap can be surpassed

```
modifier freezingLimitNotReached() {

    require(totalFreezed < FREEZE_LIMIT, "Freezing limit reached");

    _;

}



// ...

totalFreezed += _amount;
```

**Impact**

The intended freeze cap is not strictly enforced and the contract can end up in a state where `totalFreezed` exceeds `FREEZE_LIMIT`, breaking the protocol's stated limit.

**Recommendation**

Validate the post-state before incrementing, for example by requiring `totalFreezed + _amount <= FREEZE_LIMIT` (or an equivalent safe check) inside `freeze()` before updating `totalFreezed`.

**Status**

Resolved

## [M-02] MiracleWorld#claimRewards - `claimReward()` depends on `admin` being pre-funded, but this requirement is not documented/enforced

### Description

Rewards are paid by transferring tokens from admin to users, but the contract does not guarantee that admin receives tokens during initialization, and there is no documented/enforced funding requirement for the reward wallet.

### Vulnerability Details

In MiracleWorld.sol contract, initialize() mints tokens only to the wallets[] array and does not mint to admin unless admin is explicitly included in wallets[]. Later, claimReward() transfers rewards from admin to the user via _transfer(admin, msg.sender, claimRewards). If admin has insufficient token balance, claimReward() will revert, even if the user is otherwise eligible to claim. This creates an implicit dependency that admin must be funded , but the contract does not enforce or clearly document this requirement.

### Impact

If admin is not funded or runs out of MW, reward claims can fail and users may be unable to progress through the intended reward-claim and unfreeze flow.

### Recommendation

Clearly document that admin is the reward source wallet and must be sufficiently funded (and kept funded over time), and consider adding an explicit check and clearer revert reason in claimReward() when admin balance is insufficient, or redesign rewards so they are paid from a dedicated funded pool/contract rather than an implicit admin balance.

### Status

Acknowledged

## [M-03] MiracleWorld#unfreezeAmount() - unfreezeAmount() can set lastActionTime into the future, causing later calls to revert with arithmetic panic

### Description

The unfreezeAmount() allows an "instant first vest" by forcing `noOfmonths = 1` even when no time has passed, and then adds one month to lastActionTime. This can push lastActionTime ahead of block.timestamp, so the next call that subtracts timestamps underflows and reverts with a Solidity 0.8 arithmetic panic instead of the intended `UnfreezeError()`.

### Vulnerability Details

In MiracleWorld.sol contract, unfreezeAmount() computes time = block.timestamp - _freezedetail.lastActionTime and derives `noOfmonths`. When _freezedetail.principalClaimedMonth == 0 and noOfmonths == 0, it forces noOfmonths = 1 to "trigger first vest immediately", then advances _freezedetail.lastActionTime by epoctime = noOfmonths * SECONDS_IN_MONTH. If the call happens in the same block (so `time` is 0), this moves `lastActionTime` about one month into the future. On a later call before that time passes, block.timestamp - lastActionTime underflows and the transaction reverts with an arithmetic panic .

```
uint time = block.timestamp - _freezedetail.lastActionTime;

uint noOfmonths = (time * 12) / SECONDS_IN_YEAR;

if (_freezedetail.principalClaimedMonth == 0 && noOfmonths == 0) {

    noOfmonths = 1; // trigger first vest immediately

}

uint epoctime = noOfmonths * SECONDS_IN_MONTH;

_freezedetail.lastActionTime += epoctime;
```

### Impact

Users can face unexpected reverts on subsequent `unfreezeAmount()` calls, effectively soft-bricking that freeze position until timestamps catch up.

**Recommendation**

Ensure `lastActionTime` never moves into the future.

**Status**

Acknowledged

## [M-04] MiracleWorld#freeze() - `totalFreezed` never decreases, which can permanently block `freeze()` after the limit is reached

### Description

The `totalFreezed` is only incremented during `freeze()` and is never decremented during unfreezing, so freezing can become permanently unavailable once the counter reaches the limit.

### Vulnerability Details

In MiracleWorld.sol contract, `freeze()` increments both `freezedAmount[msg.sender]` and the global `totalFreezed`. During `unfreezeAmount()`, the contract reduces only `freezedAmount[msg.sender]` and does not reduce `totalFreezed`. Since `freezingLimitNotReached` requires `totalFreezed < FREEZE_LIMIT`, once `totalFreezed` reaches or exceeds `FREEZE_LIMIT`, all future calls to `freeze()` will revert forever, even if users later unlock all principal.

```
modifier freezingLimitNotReached() {

    require(totalFreezed < FREEZE_LIMIT, "Freezing limit reached");

    _;

}



function freeze(uint _amount, uint duration) external freezingLimitNotReached {

    // ...

    freezedAmount[_msgSender()] += _amount;

    totalFreezed += _amount;

}



function unfreezeAmount(uint _freezeId) external {

    // ...

    freezedAmount[_msgSender()] -= unfreezedAmount;

}
```

**Impact**

Freezing can be permanently disabled for all users after enough cumulative freezes occur, which can brick the freeze/staking feature even if all users later unfreeze.

**Recommendation**

Align `totalFreezed` with the intended meaning of the cap. If the cap is meant to limit concurrently frozen tokens, decrement `totalFreezed` when principal is unlocked and keep it consistent with the sum of currently locked amounts. If the cap is intended to be lifetime cumulative, explicitly document that design and still enforce `totalFreezed + _amount <= FREEZE_LIMIT`.

**Status**

Acknowledged

# Low

## [L-01] MiracleWorld# - Admin and owner roles can diverge causing operational lockout

### Description

The contract sets owner and admin to the same address during initialization, but later owner can change admin via setAdmin and ownership can be transferred without updating admin, so admin and owner may become different addresses and privileged actions like stopFreezing will be controlled by admin not owner..

### Recommendation

Either remove the separate admin role and use onlyOwner for admin actions, or ensure admin is always kept in sync with owner and document it.

### Status

Acknowledged

## [L-02] component#function - Use of floating pragma

**Description**

MiracleWorld.sol contract use floating pragma version (^0.8.23). It's crucial to compile and deploy contracts with the same compiler version and settings used during development and testing. Fixing the pragma version prevents compilation with different versions. Using an outdated pragma version could introduce bugs that negatively affect the contract system, while newly released versions may have undiscovered security vulnerabilities.

**Recommendation**

Change the pragma statement in contracts to a fixed version, such as pragma solidity 0.8.23;. This locks the contract to a specific compiler version.

**Status**

Acknowledged

## [L-03] MiracleWorld#SetAdmin - SetAdmin() does not emit event

**Description**

The setAdmin() function updates the admin address but does not emit any event, making admin changes harder to track and audit on-chain.

**Recommendation**

Emit an AdminUpdated(oldAdmin, newAdmin) event inside setAdmin() whenever the admin address is changed.

**Status**

Acknowledged

# Centralisation

The Miracle World project values decentralisation alongside security and utility.

The protocol's functions are designed to operate independently, minimising reliance on the internal team and hardcoded values, while maintaining robust security and functionality.

# Conclusion

After Hashlock's analysis, the Miracle World project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

# Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

**Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

**Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

# About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** hashlock.com.au
**Contact:** info@hashlock.com.au